

Homework 5

Due : Monday, May 11 at 11:59 pm

Deliverables. Submit a single PDF of your write-up to Gradescope *HW5 Write-Up*.

Start each problem on a new page.

Honor Code

Write and sign the following statement:

“I certify that all solutions in this document are entirely my own and that I have not looked at anyone else’s solution. I have given credit to all external sources I consulted.”

I certify that all solutions in this document are entirely my own and that I have not looked at anyone else's solution. I have given credit to all external sources I consulted.

1 Self-Supervised Learning for Biology

Self-supervised learning is widely used in computational biology to learn useful representations from large unlabeled datasets such as single-cell RNA sequencing (scRNA-seq), microscopy images, and protein sequences. Rather than relying on expensive human annotations (e.g., cell type labels), self-supervised learning constructs a **pre-text task** that allows the model to generate its own supervision signal.

In this problem, we will study a contrastive self-supervised objective inspired by **Contrastive Predictive Coding (CPC)** and modern contrastive learning methods such as SimCLR and CLIP. The key idea is to encourage biologically similar cells to have nearby latent representations, while unrelated cells are pushed apart.

Background: In scRNA-seq, each cell is represented by a high-dimensional gene expression vector

$$x_i \in \mathbb{R}^d,$$

where d is the number of measured genes.

An encoder maps each cell into a latent representation

$$z_i = g_{\text{enc}}(x_i) \in \mathbb{R}^m.$$

For each reference cell i , we define:

- a **positive sample** z_i^+ : a biologically similar view of the same cell (for example, a different stochastic gene dropout version of the same cell, or a nearby cell from the same local neighborhood),
- $N - 1$ **negative samples** z_j^- : latent representations from unrelated cells in the minibatch.

A context representation c_i is constructed from the reference cell representation z_i , and we define the score function

$$f(z, c_i) = \exp\left(z^\top W c_i\right),$$

where W is a learnable matrix.

The **InfoNCE (Information Noise-Contrastive Estimation)** objective is a contrastive learning loss that treats representation learning as a classification problem: among one positive sample and many negative samples, the model must identify which sample is truly related to the reference cell.

For cell i , the InfoNCE objective is

$$\mathcal{L}_i = -\log \frac{f(z_i^+, c_i)}{f(z_i^+, c_i) + \sum_{j=1}^{N-1} f(z_j^-, c_i)}.$$

The total training loss is the average over all cells in the minibatch. This objective encourages the model to assign high similarity to biologically related cells and low similarity to unrelated cells.

(a) Interpreting the Objective.

Explain why minimizing the InfoNCE objective can be interpreted as an N -way classification problem.

What are the “classes” being predicted?

(b) Gradient of the Contrastive Loss.

Let

$$s_j = z_j^\top W c_i$$

denote the similarity score between candidate cell j and the reference context c_i .

Compute

$$\frac{\partial \mathcal{L}_i}{\partial s_j}$$

for:

- (i) the positive sample
- (ii) a negative sample

and explain how these gradients encourage useful representations.

(c) Effect of More Negative Samples.

Suppose the number of negative samples N is increased substantially.

Explain mathematically why larger N often improves representation quality.

What tradeoff does this introduce during training?

(d) Contrastive Learning vs. Reconstruction.

Instead of contrastive learning, suppose we trained an autoencoder to directly reconstruct the full gene expression vector x_i .

Why might learning in latent space using a contrastive objective be preferable to directly reconstructing x_i ?

(e) Transfer Learning After Pretraining.

After self-supervised pretraining, the encoder g_{enc} is frozen and a small classifier is trained using only a small labeled dataset for a downstream task.

Explain why this transfer learning setup can outperform training a fully supervised classifier from scratch.

Why is this especially useful in settings where labeled data are limited?

(a) Minimizing the InfoNCE function can be interpreted as an N -way classification problem because, for each reference cell i , the model is represented with a set of N candidate samples: 1 positive sample z_i^+ ; $N-1$ negative sample z_i^- .

The score function $f(z, c_i) = \exp(z^\top W c_i)$ produces a non-normalized "similarity" between each candidate z and the context c_i . The denominator in the loss sums these scores over all N candidates, and the fraction is exactly the softmax over the N items.

Thus, for a fixed c_i , the model is effectively computing:

$$P[\text{candidate } k \text{ is the positive sample} | c_i] = \frac{f(z_k, c_i)}{\sum_{k=1}^N f(z_k, c_i)}$$

The loss $L_i = -\log P(\text{positive} | c_i)$ is the cross-entropy loss for a classification task with N classes.

The N classes are: 1 class for the positive sample Z_i^+ ; $N-1$ classes for each of the negative samples Z_j^- .

$$(b) \quad S_j = Z_j^T W c_i, \quad L_i = -\log \frac{f(Z_i^+, c_i)}{f(Z_i^+, c_i) + \sum_{j=1}^{N-1} f(Z_j^-, c_i)}$$

(i) For a positive sample $Z_j = Z_i^+$,

$$L_i = -\log \frac{f(S_j)}{f(S_j) + \sum_{j=1}^{N-1} f(Z_j^-, c_i)} = -\log \frac{e^{S_j}}{e^{S_j} + \sum_{i=1}^{N-1} e^{S_i^-}}, \quad \frac{\partial L_i}{\partial S_j} = \frac{\partial L_i}{\partial S_j} \left[-S_j + \log(e^{S_j} + \sum_{i=1}^{N-1} e^{S_i^-}) \right]$$

$$= -1 + \frac{e^{S_j}}{e^{S_j} + \sum_{i=1}^{N-1} e^{S_i^-}}$$

$$\text{Let } P_+ = \frac{e^{S_+}}{\sum_{k=1}^N e^{S_k}}, \quad \frac{\partial L_i}{\partial S_j} = P_+ - 1. \quad \text{Since } P_+ \leq 1, \text{ this gradient is always } \leq 0.$$

The gradient pushes S_+ up, increasing similarity between the positive sample and the reference context c_i . When P_+ is small (model is uncertain), the gradient magnitude is large, giving a strong pull to increase S_+ ; when P_+ is close to 1, the gradient approaches 0, indicating that the positive sample is already well distinguished.

$$(ii) \text{ For a negative sample } Z_j = Z_i^-, \quad \frac{\partial L_i}{\partial S_j} = \frac{e^{S_j}}{\sum_{k=1}^N e^{S_k}} = P_j.$$

The gradient pushes S_j down, decreasing similarity between the negative sample and the reference context c_i . The magnitude equals the softmax probability of the negative sample. If a negative sample is currently very similar to the context, it receives a strong push to move away; if it is already far away, the gradient is nearly zero, and the model stops wasting effort on it.

(c) A larger N makes the classification task harder because the model must distinguish the positive sample from many more distractors.

Increasing N provides a tighter approximation to the true data distribution, reduces the chances that trivial representations succeed, and forces the encoder to learn more features. The model must produce embeddings where $S_+ \gg S_j$ for many negatives. This generally improves representation quality because the latent space becomes more discriminative.

However, large N introduces costs. The model requires higher computation and memory in order to compute more pairwise similarities. It is also harder to optimize since

too many negatives can produce noisier gradients, and it takes longer training time.

cd) An autoencoder tries to reconstruct the entire gene expression vector: $x_i \rightarrow z_i \rightarrow \hat{x}_i$. This forces the latent representation to preserve nearly all information, including technical noise or uninformative gene variation. In the data, much of the raw signal may be noisy or biologically irrelevant.

By contrast, a contrastive objective focuses on semantic similarity. Biologically related cells are grouped together, while unrelated cells are separated. Therefore the encoder learns features that are invariant to noise and perturbations and more biologically meaningful.

Reconstruction emphasizes fidelity to raw input, but contrastive learning emphasizes informative structure in latent space.

ce) This can outperform training from scratch because the pretrained encoder already contains useful representations such as cell-type structure, gene co-expression patterns or biological trajectories. The downstream classifier therefore starts with strong features instead of learning everything from random initialization.

Self-supervised learning leverages abundant unlabeled data to learn representations beforehand. As a result, fewer labeled examples are needed, overfitting is reduced and the generalization improves. This is particularly useful in computational biology, where labeled data are often expensive and difficult to annotate.

Diffusion Models

1. Diffusion Model Theory

Consider a data distribution and the denoising diffusion model $x_\sigma = x_0 + \sigma\epsilon$ with noise $\epsilon \sim \mathcal{N}(0, I)$.

- (a) Show that for random variables X and Y , the function $f(y)$ that minimizes $\mathbb{E}_{x,y} \|f(y) - X\|^2$ is the conditional expectation, i.e.,

$$f(y) = \mathbb{E}[X \mid Y = y].$$

- (b) Using part (a), derive the optimal denoising function $\epsilon_\sigma^*(x_\sigma)$ that minimizes

$$L_\sigma(\epsilon_\sigma) = \mathbb{E}_{x_0, \epsilon} \|\epsilon_\sigma(x_0 + \sigma\epsilon) - \epsilon\|^2$$

Express $\epsilon_\sigma^*(x_\sigma)$ in terms of the data density and the observed noisy input.

- (c) Let x_0 follow a continuous density $p_0(x)$ and define a time-varying noise schedule $x_t = x_0 + \sigma(t)\epsilon$. Define the vector field $v(x, t) = \mathbb{E}\left[\frac{d}{dt}\sigma(t)\epsilon \mid x_t = x\right]$ and the time-evolving density $p_t(x)$. Show that they satisfy the continuity equation

$$\partial_t p_t(x) + \nabla \cdot (v(x, t) p_t(x)) = 0$$

$$c_a) \mathbb{E}_{x,y} \|f(y) - x\|^2 = \mathbb{E}_y [\mathbb{E}_{x|Y} \|f(y) - x\|^2]$$

To minimize $\mathbb{E}_{x,y} \|f(y) - x\|^2$, we only need to minimize $\mathbb{E}_{x|Y=y} \|f(y) - x\|^2$

$$J(f(y)) = \mathbb{E} [\|f(y) - x\|^2 \mid Y=y]$$

$$= \|f(y)\|^2 - 2f(y)^T \mathbb{E}[x \mid Y=y] + \mathbb{E}[\|x\|^2 \mid Y=y]$$

$$\frac{\partial J(f(y))}{\partial f(y)} = 2f(y) - 2\mathbb{E}[x \mid Y=y] = 0$$

$$f(y) = \mathbb{E}[x \mid Y=y]$$

$$b) L_\sigma(\epsilon_\sigma) = \mathbb{E}_{x_0, \epsilon} \|\epsilon_\sigma(x_0 + \sigma\epsilon) - \epsilon\|^2 = \mathbb{E}_{x_0, \epsilon} \|\epsilon_\sigma(x_0) - \epsilon\|^2$$

$$\epsilon_\sigma^*(x_\sigma) = \mathbb{E}[\epsilon \mid x_\sigma], \quad x_\sigma = x_0 + \sigma\epsilon, \quad \epsilon = \frac{x_\sigma - x_0}{\sigma}$$

$$\epsilon_\sigma^*(x_\sigma) = \mathbb{E}\left[\frac{x_\sigma - x_0}{\sigma} \mid x_\sigma\right] = \frac{1}{\sigma} (x_\sigma - \mathbb{E}[x_0 \mid x_\sigma])$$

$$p(x_0 \mid x_\sigma) = \frac{p(x_\sigma \mid x_0) \cdot p(x_0)}{p(x_\sigma)} = \frac{p(x_0) \mathcal{N}(x_\sigma; x_0, \sigma^2 I)}{\int p(x_0) \cdot \mathcal{N}(x_\sigma; x_0, \sigma^2 I) dx_0}$$

$$E[X_0 | X_6] = \int p(X_0 | X_6) \cdot X_0 dX_0 = \frac{\int X_0 p(X_0) N(X_6; X_0, \sigma^2 I) dX_0}{\int p(X_0) N(X_6; X_0, \sigma^2 I) dX_0}$$

$$\text{Thus, } \xi_\sigma^*(X_6) = \frac{1}{\sigma} (X_0 + \sigma \xi - \frac{\int X_0 p(X_0) N(X_6; X_0, \sigma^2 I) dX_0}{\int p(X_0) N(X_6; X_0, \sigma^2 I) dX_0})$$

$$(c) \quad X_t = X_0 + \sigma(t) \xi, \quad v(X, t) = E\left[\frac{d}{dt} \xi(t) \xi | X_t = x\right]$$

$$\frac{d}{dt} X_t = \frac{d}{dt} \sigma(t) \xi$$

Consider any smooth test function $\phi(x)$, then

$$\frac{d}{dt} E[\phi(X_t)] = \frac{d}{dt} \int \phi(x) p_t(x) dx = E\left[\frac{d}{dt} \phi(X_t)\right]$$

$$\frac{d}{dt} \phi(X_t) = \nabla \phi(X_t)^T \frac{d}{dt} X_t,$$

$$\begin{aligned} \frac{d}{dt} E[\phi(X_t)] &= E\left[\nabla \phi(X_t)^T \frac{d}{dt} X_t\right] \\ &= E\left[\nabla \phi(X_t)^T \frac{d}{dt} \sigma(t) \xi\right] \end{aligned}$$

Condition on $X_t = x$

$$E\left[\nabla \phi(X_t)^T \frac{d}{dt} \sigma(t) \xi\right] = E\left[E\left[\nabla \phi(X_t)^T \frac{d}{dt} \sigma(t) \xi | X_t = x\right]\right]$$

$$= E\left[\nabla \phi(X_t)^T E\left[\frac{d}{dt} \sigma(t) \xi | X_t = x\right]\right] = E\left[\nabla \phi(X_t)^T v(X, t)\right]$$

$$= \int \nabla \phi(x)^T v(x, t) p_t(x) dx = - \int \phi(x) \nabla \cdot (v(x, t) p_t(x)) dx$$

$$\text{On the other hand, } \frac{d}{dt} E[\phi(X_t)] = \int \phi(x) \frac{\partial}{\partial t} p_t(x) dx$$

$$\text{So } \int \phi(x) \frac{\partial}{\partial t} p_t(x) dx = - \int \phi(x) \nabla \cdot (v(x, t) p_t(x)) dx$$

$$\int \phi(x) [\frac{\partial}{\partial t} p_t(x) + \nabla \cdot (v(x, t) p_t(x))] dx = 0$$

Since this holds for all test function ϕ ,

$$\frac{\partial}{\partial t} p_t(x) + \nabla \cdot (v(x, t) p_t(x)) = 0$$

Flow Matching with General Interpolation Paths

Background. In flow matching, one typically defines a path between a noise sample $x_0 \sim \mathcal{N}(0, I)$ and a data sample x_1 using linear interpolation:

$$x_t = t x_1 + (1 - t) x_0.$$

In this case, the ground-truth velocity is constant:

$$\dot{x}_t = x_1 - x_0.$$

A natural generalization is to define a time-dependent interpolation:

$$x_t = \alpha(t) x_1 + \beta(t) x_0,$$

where $\alpha(t)$ and $\beta(t)$ are differentiable functions satisfying

$$\alpha(0) = 0, \quad \beta(0) = 1, \quad \alpha(1) = 1, \quad \beta(1) = 0.$$

This ensures that the path starts at x_0 and ends at x_1 .

(a) General velocity field

Derive an expression for the ground-truth velocity $\dot{x}_t$ along this path in terms of $\alpha'(t)$ and $\beta'(t)$.

Show that for the linear choice $\alpha(t) = t$, $\beta(t) = 1 - t$, your expression reduces to the constant velocity

$$\dot{x}_t = x_1 - x_0.$$

(b) Single-function parameterization

Suppose instead that the path is written using a single function $h(t)$:

$$x_t = h(t) x_1 + (1 - h(t)) x_0$$

where $h(t)$ is differentiable, monotonically increasing, and satisfies $h(0) = 0$, $h(1) = 1$.

(i) Show that the velocity can be written as

$$\dot{x}_t = h'(t) (x_1 - x_0).$$

(ii) Explain how the choice of $h(t)$ affects the *speed* at which the path moves from x_0 to x_1 .

(iii) Compare the behavior of the following choices:

$$h(t) = t, \quad h(t) = t^2, \quad h(t) = \sin\left(\frac{\pi t}{2}\right).$$

In particular, describe whether the trajectory moves faster near $t = 0$ or $t = 1$ for each case.

$$(a) \quad X_t = \alpha(t)X_1 + \beta(t)X_0$$

$$\dot{X}_t = \alpha'(t)X_1 + \beta'(t)X_0$$

$$\alpha(t)=t, \alpha'(t)=1; \beta(t)=1-t, \beta'(t)=-1$$

$$\dot{X}_t = X_1 - X_0$$

$$(b) (i) \quad X_t = h(t)X_1 + (1-h(t))X_0$$

$$\dot{X}_t = h'(t)X_1 - h'(t)X_0$$

$$= h'(t)(X_1 - X_0)$$

(ii) The direction of motion is always $X_1 - X_0$.

However, the speed is scaled by $h'(t)$. Larger $h'(t)$ implies higher speed, so $h(t)$ controls how quickly the interpolation progresses over time.

If $h'(t)$ is larger near $t=0$, the trajectory moves quickly at the beginning and slows later. If $h'(t)$ is larger near $t=1$, the trajectory starts slowly and speeds up near the end.

$$(iii) \quad h(t)=t, h'(t)=1, \dot{X} = X_1 - X_0.$$

The speed is constant throughout time. Same speed near $t=0$ and $t=1$.

$$h(t)=t^2, h'(t)=2t, \dot{X} = 2t(X_1 - X_0).$$

Near $t=0$, $h'(t) \approx 0$, so motion is very slow.

Near $t=1$, $h'(t) \approx 2$, so motion is much faster.

It starts slowly, accelerates over time and moves faster near $t=1$.

$$h(t) = \sin\left(\frac{\pi t}{2}\right), h'(t) = \frac{\pi}{2} \cos\left(\frac{\pi}{2}t\right), \dot{X} = \frac{\pi}{2} \cos\left(\frac{\pi}{2}t\right)(X_1 - X_0)$$

Near $t=0$, $h'(t) \approx \frac{\pi}{2}$, which is the maximum speed.

Near $t=1$, $h'(t) \approx 0$, speed goes to 0.

It moves faster near $t=0$ and gradually slows down.